

帶有全有或全无产出模式及非零提前期的库存问题的启发式与深度强化学习方法

(Heuristics and deep reinforcement learning for the inventory problem with an all-or-nothing yield pattern and non-zero leadtimes)

Yuting Yan, Frank (Youhua) Chen, Zhe Fu, Wenjie Bi

Computers & Operations Research 186 (2026) 107305

研究背景与系统设定 (Background & Problem Setting)

核心场景：单产品、定期盘点 (Periodic-review) 的库存系统。

关键特征：随机需求 (Stochastic demand)，非零提前期 ($L \geq 0$)，以及全有或全无随机产出模式 (All-or-nothing random yield pattern)。

产出机制：订单到达时必须进行抽样检测。如果检测失败，整批货物将被直接报废或退回。

优化目标：寻找最优订货策略，以最小化总预期折扣成本（包含采购、库存持有、缺货惩罚）。

[Insight / 通俗解释]

传统模型多假设产出比例是随机的（例如到了80%的货），但本研究严格限制为‘要么100%合格，要么0%全部报废’的极端物理场景。这在生物制药行业的合规质检中极为常见。

模型构建：变量与系统动态 (Model Formulation: Variables & System Dynamics)

[Variables Definition]	
T	规划期视界 (Horizon)
L	提前期 (Leadtime, $L \geq 0$)
Y_t	第 t 期的随机产出变量。 $Y_t = 1$ 的概率为 p ; $Y_t = 0$ 的概率为 $1 - p$ 。
I_t	第 t 期的库存水平 (正数为可用库存, 负数为缺货量)。
$\mathbf{Q}_t$	在途订单向量 (Pipeline orders) , $\mathbf{Q}_t = (q_{t-1}, q_{t-2}, \dots, q_{t-L+1})$ 。

[System Dynamics Equation]
$I_{t+1} = I_t + y_{t+1}q_{t+1-L} - d_t$
其中 d_t 为第 t 期观测到的需求, $y_{t+1}q_{t+1-L}$ 为 L 期前发出并在 $t + 1$ 期检测合格的可用数量。

系统的下一个状态不仅取决于当前的库存 I_t , 还严重依赖于一个 $L - 1$ 维的历史在途订单向量 $\mathbf{Q}_t$ 。这种历史依赖性直接导致了后续极其棘手的“状态空间爆炸”问题。

模型构建：成本与目标函数 (Model Formulation: Cost Function)

Step 1: 单期持有与缺货惩罚成本

$$H(I_t) = c_h[I_t - d_t]^+ + c_b[d_t - I_t]^+$$

(c_h : 单位持有成本, c_b : 单位缺货成本, $[x]^+ = \max\{x, 0\}$)



Step 2: 第 t 期的实际发生总成本

$$c_t(I_t, q_{t-L}) = c_o q_{t-L} y_t + H(I_t)$$

(c_o : 单位采购成本。仅在货物送达且检验合格 $y_t = 1$ 时才支付)



Step 3: 全局目标函数 (Global Objective)

$$\min_{q_t \geq 0} \mathbb{E} \left[\sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L}) \right]$$

马尔可夫决策过程与贝尔曼方程 (MDP Formulation & Bellman Equation)

状态空间 (State Space): $s_t = (I_t, \mathbf{Q}_t) \in \mathcal{S}$

动作空间 (Action Space): $q_t \in \mathcal{A}$

[Dynamic Programming Box]

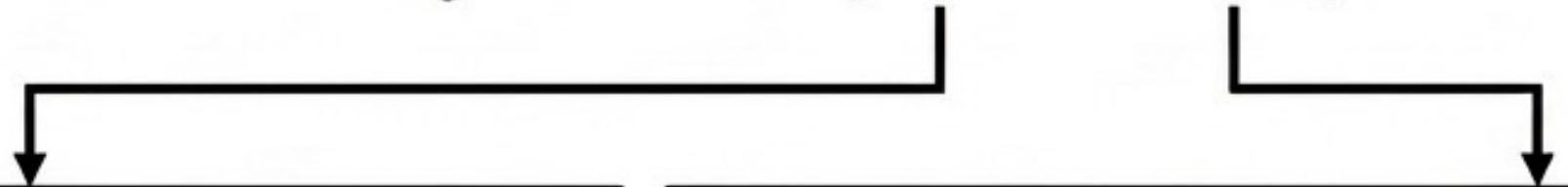
$$\begin{aligned} V_t(I_t, \mathbf{Q}_t) &= \min_{q_t \geq 0} \mathbb{E}[c_o q_t Y_{t+L} + \alpha^L H(I_{t+L}) \\ &\quad + \alpha V_{t+1}(I_t + Y_{t+1} q_{t+1-L} - D_t, \mathbf{Q}_{t+1}, \mathbf{Q}_{t+1})] \\ &\triangleq \min_{q_t \geq 0} \{F_t(I_t, \mathbf{Q}_t, q_t)\} \end{aligned}$$

$$\text{其中 } I_{t+L} = I_t + \sum_{i=1}^L Y_{t+i} q_{t+i-L} - D_{[t, t+L-1]}.$$

基准策略 1：线性通胀规则

(Benchmark 1: Linear Inflation Rule - LIR)

[Policy Equation]

$$q_t^{LIR} = \beta [S_L - IP_t]^+$$


当前库存位置 (Inventory Position):

$$IP_t = I_t + p \sum_{i=1}^{L-1} q_{t-i}$$

目标库存水平 (Target Level):

$$S_L = (L+1)\mu_D + k\sqrt{(L+1)\sigma_D^2 + \max\{L, 1\} \frac{1-p}{p} (\mu_D^2 + \sigma_D^2)}$$

基准策略 2: 加权报童启发式

(Benchmark 2: Weighted Newsvendor Heuristic - WNH)

Logic: 枚举在途订单所有 2^{L-1} 种可能的存活/报废路径集合 $\mathcal{O}$ 。

[Policy Equation]

$$q_t^{WNH} = \sum_{o \in \mathcal{O}} \left\{ \underbrace{p^{\sum y_i} (1-p)^{L-1-\sum y_i}}_{\text{该特定存活路径发生的固定概率 (静态权重)}} \left[\underbrace{S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L})}_{\text{在该特定路径发生时, 为了达到目标库存, 所需的“理想订货量”}} \right]^+ \right\}$$

该特定存活路径发生的
固定概率 (静态权重)

在该特定路径发生时, 为了达到
目标库存, 所需的“理想订货量”

创新方法 1：梯度下降搜索启发式 (Gradient Descent Search Heuristic - GDSH)

Motivation: WNH 使用固定的概率作为权重，且目光短浅 (Myopic)。GDSH 则将这些权重视为可在整个规划期内全局优化的参数。

WNH (Benchmark 2) - Fixed Weights

$$\dots p^{\sum y_i} (1 - p)^{L-1-\sum y_i} \dots$$

Static Weight w_j
(Fixed Probability)

Evolution:
Fixed Prob. $\rightarrow$
Optimizable Params

[Policy Equation]

$$q_t^{GDSH} = \sum_{o \in \mathcal{O}} \left\{ w_j \left[S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L}) \right]^+ \right\},$$
$$j = 1 + \sum_{i=1}^{L-1} 2^{i-1} y_i$$

Weight Vector: $\mathbf{w} = (w_1, \dots, w_{2^{L-1}})$ 。注意：此处的权重向量是状态无关的 (State-independent) 且时间无关的 (Time-independent)。

GDSH 的核心数学性质：凸性 (Mathematical Properties of GDSH)

[Proposition 1 / 命题 1]

对于公式 (5) 所表达的策略 q_t^{GDSH} ，目标函数 $F_t(I_t, \mathbf{Q}_t, q_t^{\text{GDSH}})$ 在任意时间 t 对权重向量 $\mathbf{w}$ 都是凸函数 (Convex)。

[Proof Logic / 证明逻辑]

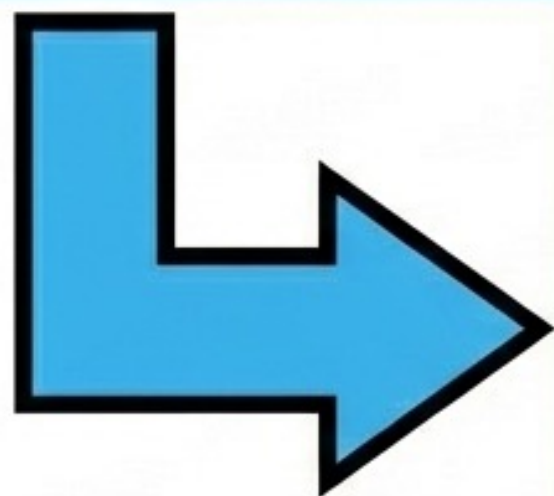
1. 已知 F_t 针对 q_t^{GDSH} 是凸的。
2. 凸函数与仿射函数 (Affine function) 的复合依然是凸函数。
3. 由于公式 (5) 中 q_t^{GDSH} 对 $\mathbf{w}$ 呈线性关系，因此 F_t 对 $\mathbf{w}$ 保持凸性。■

从数学上保证了我们可以安全地使用梯度下降算法 (Gradient Descent) 在整个参数空间中找到全局最优的权重向量 $\mathbf{w}^*$ ，而不会陷入局部最优。

终极解决方案：深度强化学习架构

(The DRL Approach - State-Dependent Weights)

Limitation of GDSH: GDSH 的权重 $\mathbf{w}$ 依然是静态的。但在现实中，最优策略显然应该随着当前库存 I_t 和在途订单 $\mathbf{Q}_t$ 的变化而动态调整。



[DRL Policy Structure]

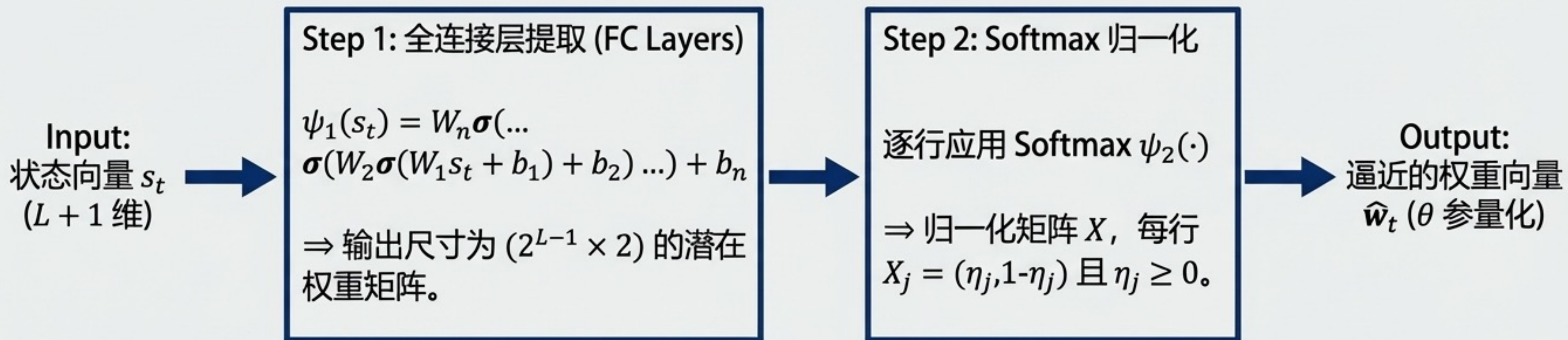
$$q_t^{DRL} = \sum_{o \in \mathcal{O}} \left\{ w_j^t \left[S_L - I_t - \sum_{o \in \mathcal{O}} \left\{ w_j \left[S_L - I_t - \sum_{i=1}^{L-1} (y_i q_{t+i-L}) \right]^+ \right\} \right] \right\},$$

矩阵重写形式: $q_t^{DRL} = (\mathbf{s}_t \mathbf{A})^+ \mathbf{w}_t$

[Corollary 1 / 推论 1]

$F_t(I_t, \mathbf{Q}_t, q_t^{DRL})$ 对动态权重 $\mathbf{w}_t$ 依然保持凸性。

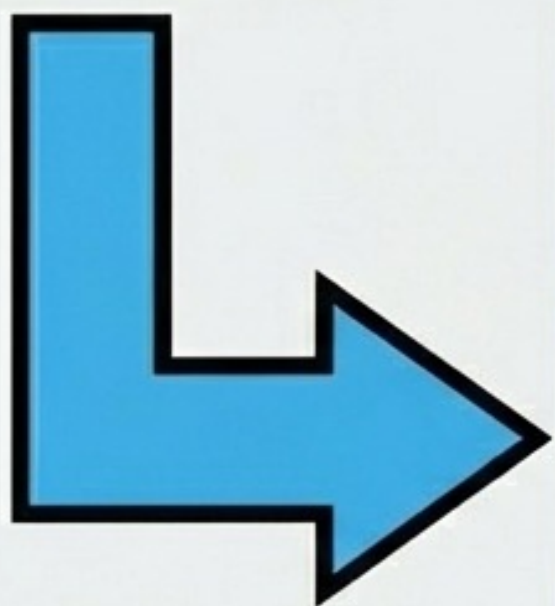
DRL 神经网络架构解构：特征提取与归一化 (Neural Network Architecture - Part 1)



神经网络在这里扮演高级特征提取器。前两步通过多层仿射变换与 ReLU 非线性激活，将复杂的库存和订单状态压缩、提炼，并将其严格归一化为满足概率定义的标准系数。

DRL 神经网络架构解构：动作空间边界约束 (Action Space Bounding)

Teacher Policy Anchor: WNH 作为一个优质的近视解，其输出的权重 $\mathbf{w}_{WNH}$ 被指定为神经网络搜索的“导师坐标”。



[Set Constraint / 边界约束]

$$\mathcal{V} = \{\hat{w}_j^t : w_{WNH,j} - \delta \leq \hat{w}_j^t \leq w_{WNH,j} + \delta\}$$

Step 3: 边界映射函数 (Mapping Function)

$$\psi_3(X) = (X \odot U) \cdot [1, 1]^T$$

其中界限矩阵 $U = [\mathbf{w}_{WNH} - \delta, \mathbf{w}_{WNH} + \delta]$ 。



最终策略函数 (Final Policy Function):

$$\zeta_{\theta}(s_t) = \psi_3 \circ \psi_2 \circ \psi_1(s_t) \Rightarrow q_t^{DRL} = (s_t A)^+ \zeta_{\theta}(s_t)$$

DRL 目标函数设计：导师策略混合与奖励塑造 (Blending Teacher Policy - Loss Function)

[Loss Function Definition]

$$\mathcal{L} = \exp \left(\frac{C_T^{DRL} - C_T^{WNH}}{C_T^{WNH}} \times 100 \right)$$

Where:

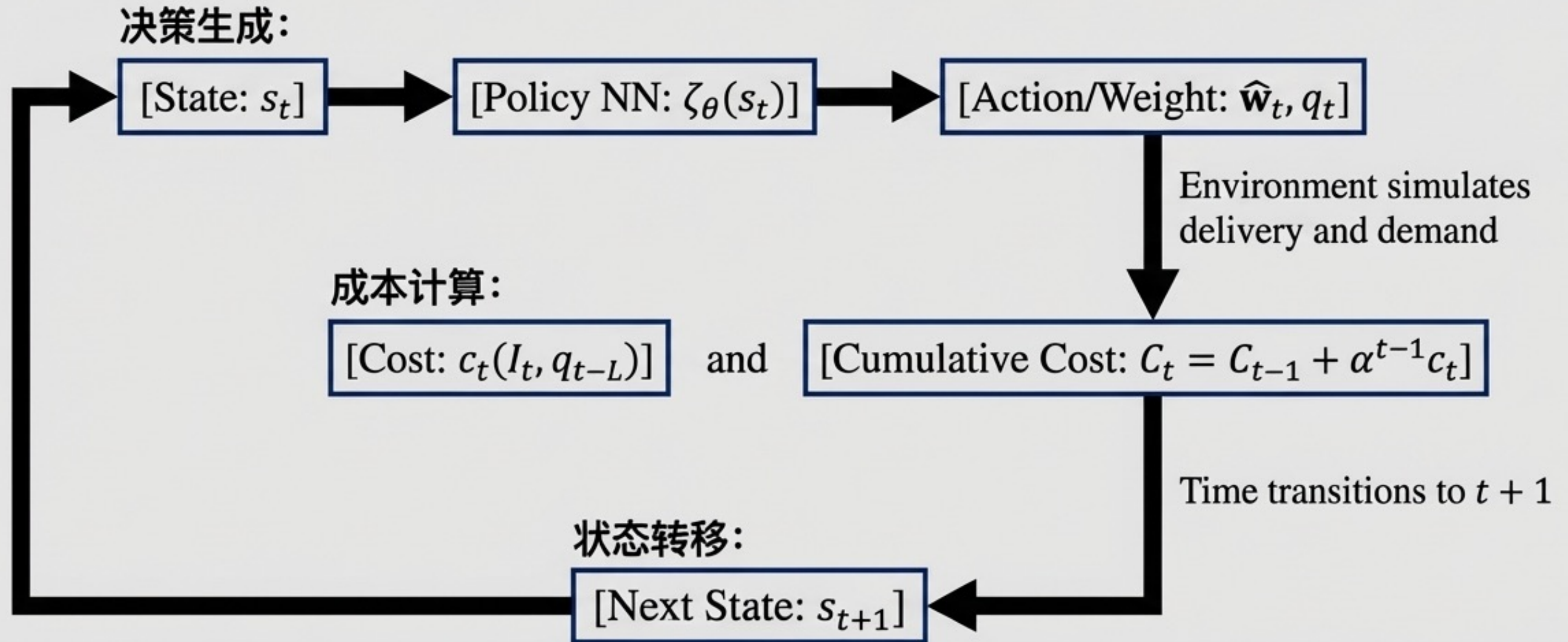
$$C_T^{DRL} = \sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L}^{DRL})$$

(神经网络策略在整条轨迹上的总成本)

$$C_T^{WNH} = \sum_{t=1}^T \alpha^{t-1} c_t(I_t, q_{t-L}^{WNH})$$

(导师策略在同一轨迹上的总成本)

智能体与环境交互动态 (System Interaction Dynamics in Period t)



DRL 批量梯度下降训练算法 (Algorithm 1: Training the DRL Agent)

[Algorithm 1]

- 1: Initialize policy network $\zeta_{\theta}(\cdot)$ with parameters θ .
- 2: **repeat**
- 3: **for** each episode **do**
- 4: Set initial inventory = 0, calculate S_L , C_T^{WNH} , search bounds U .
- 5: **for** $t = 1 : T$ **do**
- 6: Obtain weight $\hat{\mathbf{w}}_t = \zeta_{\theta}(s_t)$.
- 7: Calculate action $q_t = (s_t A)^+ \hat{\mathbf{w}}_t$.
- 8: Execute q_t , observe c_t and s_{t+1} . Update cumulative cost C_t .
- 9: **end for**
- 10: **end for**
- 11: Calculate Loss $\mathcal{L}$, perform batch gradient descent, update θ .
- 12: **until** maximum training epochs reached.